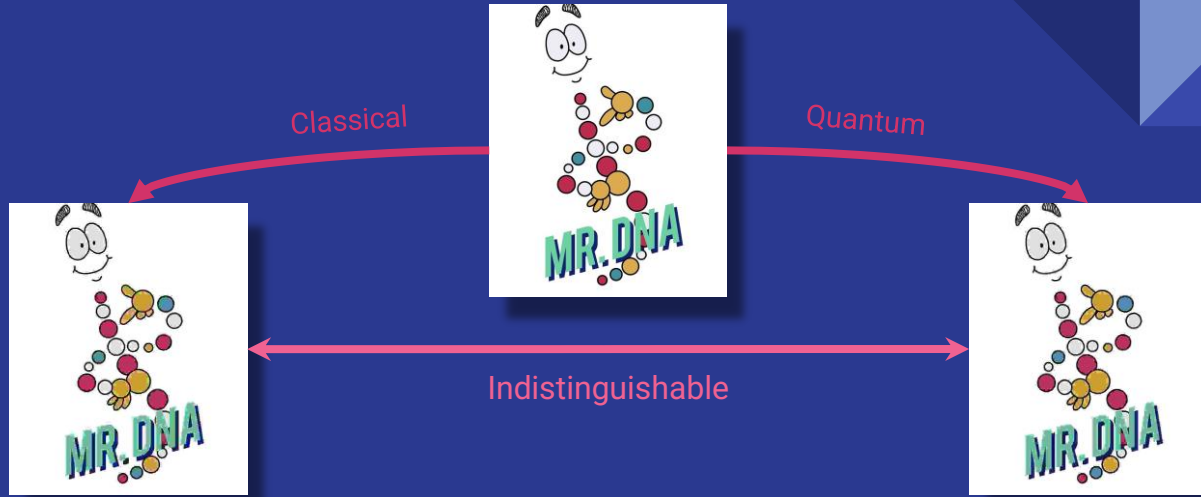




Quantum Implementation of Classic JPEG Compression

By Alex Dufresne-Nappert, Ryan Becze, Ty Davis

Historical Context



Dr. Nasir Ahmed

- Consumer introduction to digital photography - available in the late 1990s [1]
- Barrier #1 - Quality of digital limited to sensor capabilities
- Barrier #2 - New sensor capabilities limited by storage capabilities
- Solution - Using image compression to reduce the required data for an image.
- By taking “...advantage of visual perception and the statistical properties of image data...” noise and other high frequency data can be removed without impacting human-defined quality [2]
- In preparation of adoption the JPEG was created and patented by Dr. Nasir Ahmed in 1992 [3]

The widespread adoption of the JPEG format and its subsequent iterations allowed for the rapid adoption of digital cameras. Marking a pivotal moment in the evolution of the digital age.

Breakdown of JPEG Compression Algorithm

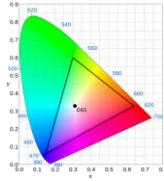
General Process Overview

High level:

Decompose an image into frequency components, and remove high-frequency details that the human eye typically ignores.

Colour Space Transformation

Transform the normal RGB pixel values into colour space



*Downsampling**

Optional Allows for significant data reduction beforehand using a flat ratio

Block Splitting

Make a 8x8 grid of each colour space parameter

Frequency Transform

Transform these matrices segmented blocks into their frequency space

Special Quantization Matrix

Divide by a standardized matrix to remove high frequency

Encoding

Encode the data using an encoding algorithm

Colour Space Transformation

Lossy*



$$\begin{aligned} R_0 &= Y' - 0.344136(Cb - 128) + 1.402(Cr - 128) \\ G_0 &= Y' - 0.58135(Cb - 128) - 0.41868(Cr - 128) \\ B_0 &= Y' + 1.1772(Cb - 128) - 0.174478(Cr - 128) \end{aligned}$$



Raw digital image RGB

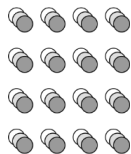
Parameterize

Digital image in colour space

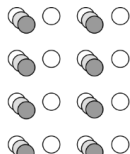
Segmented colour space layers: Y, Cb, Cr data respectively

Downsampling

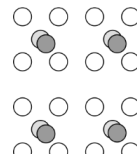
- Optional pre-transformation step.
- Lack of cone density in human eyes
- Chrominance can be reduced
- (traditionally 4:2:0) [4,5]



4:4:4



4:2:2



4:2:0



4:1:1



4:2:0



4:2:2

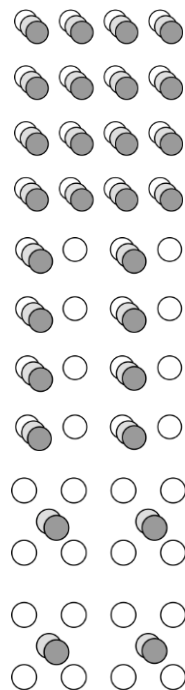


4:4:4



Block Splitting

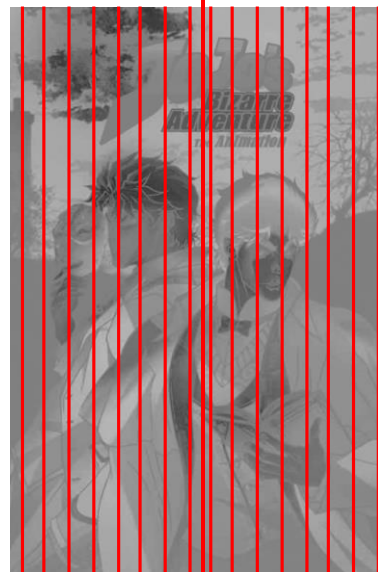
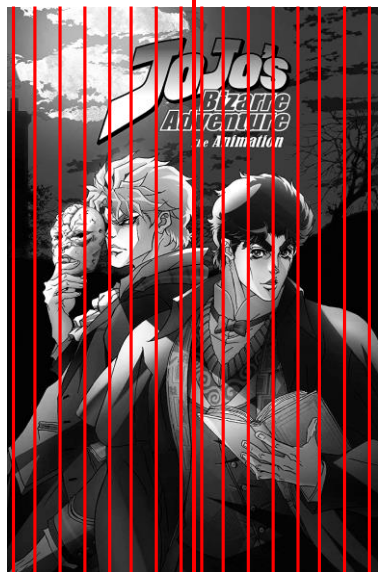
Lossless



4:4:4

4:2:2

4:2:0



Reflect on subsampling ratio [3]

Split values into blocks (MCU)

Pixel value shift to center 0
[0,255] → [-128,127]

Frequency Transform

Lossless

$$\begin{matrix}
 & \xrightarrow{x} \\
 \begin{bmatrix} 52 & 55 & 61 & 66 & 70 & 61 & 64 & 73 \\ 63 & 59 & 55 & 90 & 109 & 85 & 69 & 72 \\ 62 & 59 & 68 & 113 & 144 & 104 & 66 & 73 \\ 63 & 58 & 71 & 122 & 154 & 106 & 70 & 69 \\ 67 & 61 & 68 & 104 & 126 & 88 & 68 & 70 \\ 79 & 65 & 60 & 70 & 77 & 68 & 58 & 75 \\ 85 & 71 & 64 & 59 & 55 & 61 & 65 & 83 \\ 87 & 79 & 69 & 68 & 65 & 76 & 78 & 94 \end{bmatrix} & \xrightarrow{y} \\
 g = & \begin{bmatrix} -76 & -73 & -67 & -62 & -58 & -67 & -64 & -55 \\ -65 & -69 & -73 & -38 & -19 & -43 & -59 & -56 \\ -66 & -69 & -60 & -15 & 16 & -24 & -62 & -55 \\ -65 & -70 & -57 & -6 & 26 & -22 & -58 & -59 \\ -61 & -67 & -60 & -24 & -2 & -40 & -60 & -58 \\ -49 & -63 & -68 & -58 & -51 & -60 & -70 & -53 \\ -43 & -57 & -64 & -69 & -73 & -67 & -63 & -45 \\ -41 & -49 & -59 & -60 & -63 & -52 & -50 & -34 \end{bmatrix}
 \end{matrix}$$

$$G_{u,v} = \frac{1}{4} \alpha(u) \alpha(v) \sum_{x=0}^7 \sum_{y=0}^7 g_{x,y} \cos \left[\frac{(2x+1)u\pi}{16} \right] \cos \left[\frac{(2y+1)v\pi}{16} \right]$$

u = frequency v = spatial

Alpha is Normalization Factors

Example matrix

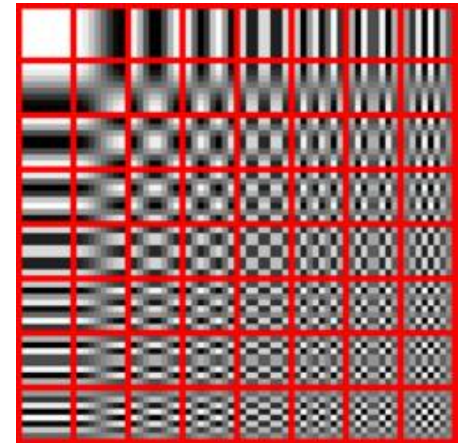
Example with aforementioned shift

Apply discrete cosine transform (2D-DCT)

$$\begin{matrix}
 & \xrightarrow{u} \\
 G = & \begin{bmatrix} -415.38 & -30.19 & -61.20 & 27.24 & 56.12 & -20.10 & -2.39 & 0.46 \\ 4.47 & -21.86 & -60.76 & 10.25 & 13.15 & -7.09 & -8.54 & 4.88 \\ -46.83 & 7.37 & 77.13 & -24.56 & -28.91 & 9.93 & 5.42 & -5.65 \\ -48.53 & 12.07 & 34.10 & -14.76 & -10.24 & 6.30 & 1.83 & 1.95 \\ 12.12 & -6.55 & -13.20 & -3.95 & -1.87 & 1.75 & -2.79 & 3.14 \\ -7.73 & 2.91 & 2.38 & -5.94 & -2.38 & 0.94 & 4.30 & 1.85 \\ -1.03 & 0.18 & 0.42 & -2.42 & -0.88 & -3.02 & 4.12 & -0.66 \\ -0.17 & 0.14 & -1.07 & -4.19 & -1.17 & -0.10 & 0.50 & 1.68 \end{bmatrix} \\
 & \downarrow v.
 \end{matrix}$$

Image data coefficients in frequency space (Y, C_B, C_R)

The frequency space is broken into an array of common frequency patterns [3]



Quantization

Lossy

$$Q = \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}$$

$$B_{j,k} = \text{round} \left(\frac{G_{j,k}}{Q_{j,k}} \right) \text{ for } j = 0, 1, 2, \dots, 7; k = 0, 1, 2, \dots, 7$$

$$B = \begin{bmatrix} -26 & -3 & -6 & 2 & 2 & -1 & 0 & 0 \\ 0 & -2 & -4 & 1 & 1 & 0 & 0 & 0 \\ -3 & 1 & 5 & -1 & -1 & 0 & 0 & 0 \\ -3 & 1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

ISO-Standard quantization matrix [3]

New quantized Discrete Cosine Transform coefficients

Unquantized (Q=0)



Y



Cb



Cr

Quantized (Q=0)



Y

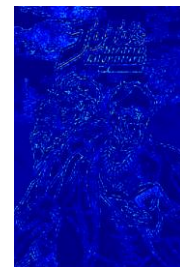


Cb

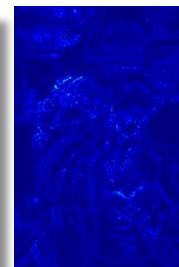


Cr

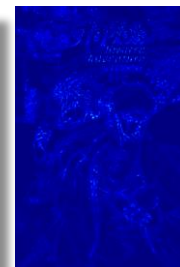
Heat Map Difference *(Q=100)*



Y



Cb

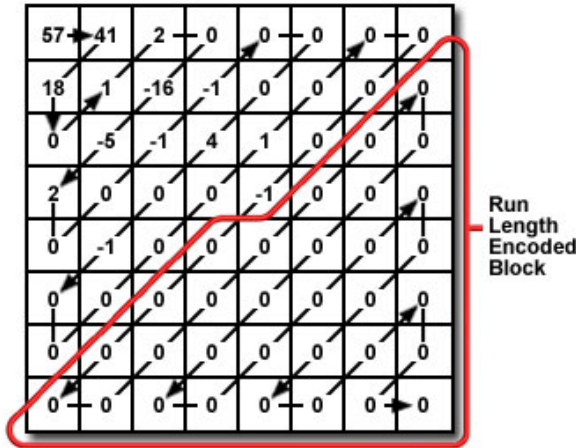


Cr

Encoding

Lossless

Run Length Encoding of DCT Blocks



Zigzag Scan Order

Figure 1

of zeros before upcoming digit

[57, 41, 18, 0, 1, 2, 0, -16, -5, 2, 0,
0, -1, -1, 0, 0, 0, 4, 0, -1, 0, 0, 0,
0, 1, 0, 0, 0, 0, 0, -1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]



[(0,57), (0,41), (0,18), (1,1), ... ,
(1,-1), (5,1), (5,-1), EOB]

Symbol	Frequency	Code
EOB	very common	0
(0,0)	common	1
(0,1)	less common	110
(3,4)	rare	11101

...

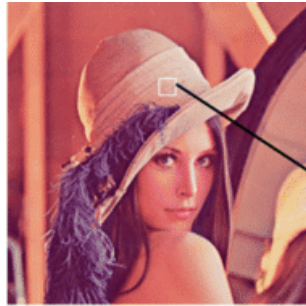
Zig-Zag encode to order frequency from high to low [6]

RLE pair output vector

Huffman encoding

Write the manipulated data to JPEG file format = Image compressed!

Process Recap



Luminance
Component

173	184	190	198	194	192	192	192
180	187	182	189	193	191	194	196
180	180	181	182	184	189	196	199
178	174	174	175	180	186	175	177
170	171	176	184	185	175	172	181
173	167	170	176	178	181	179	177
165	167	165	168	174	171	175	181
162	163	163	158	162	169	172	175

DCT
Transformation

1430	-32	-4	-1	4	-4	0	-2
64	-2	-9	-4	-1	0	-3	-3
0	-7	3	-6	-6	0	-4	0
7	3	-16	-3	5	1	-2	-1
-12	8	-8	-2	-1	-1	4	-1
-3	0	0	2	-2	0	2	0
3	-4	2	0	-4	6	3	0
5	-4	-1	-10	4	-1	-3	2

JPEG Quantization Table

8	16	19	22	26	27	29	34
16	16	22	24	27	29	34	37
19	22	26	27	29	34	34	38
22	22	26	27	29	34	37	40
22	26	27	29	32	35	40	48
26	27	29	32	35	40	48	58
26	27	29	34	38	46	56	69
27	29	35	38	46	56	69	83

Quantization

179	-2	0	0	0	0	0	0
4	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	-1	0	0	0	0	0
-1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

The Case for Quantum

Why Use A Quantum Algorithm?

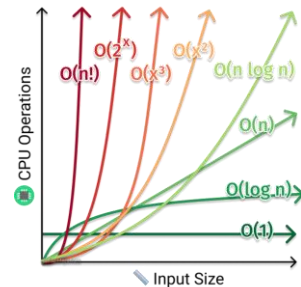
JPEG is the standard for image compression in modern applications

JPEG relies on Discrete Cosine Transform (DCT), a frequency space transform which has precedent in quantum systems through QFTs

Classical DCTs are very efficient but are limited by their $N \log_2 N$ scaling

Qubits could tangentially calculate these recurring 8x8 calculations

Standard QFT potentially scales as a $(\log_2 N)^2$ in gate count [7]



Quantum Implementation

Proposed:

Replace the classical with a Quantum Discrete Cosine Transform (QDCT)

Discrete Cosine Transform → Quantum Discrete Cosine Transform

Trade-Off:

Qubits require state preparation in addition to typical pixel preparation

QDCT outputs probabilistic results → requires repeated measurements

Question:

Does Quantum JPEG processing hold merit?

Implementing Quantum JPEG Compression

Encoding To Quantum States

1. Prepare the 8x8 blocks for ancilla dimensions → 16x16

```
# Create Padded blocks so that Ancilla Qubit Data is populated
blocks16x16 = np.zeros((numBlocks, 16, 16), dtype=np.float32)
blocks16x16[:, :8, :8] = blocks8x8
```

2. Normalize the incoming colour space (YCrCb) data

```
norm = np.linalg.norm(flattenedBlocks)
normalizedData = flattenedBlocks / norm if norm != 0 else flattenedBlocks
```

3. Initialize quantum state using state vector

$$|\psi_{pixels}\rangle = \sum_{i=0}^{63} c_i |i\rangle$$

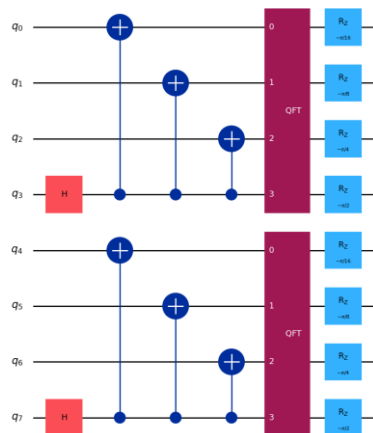
```
# Initialize State and Circuit
state = Statevector(normalizedData)
```


Mapping our Quantum Circuit

4. Creating Quantum Circuit

```
numQubits = int(np.log2(len(flattenedBlocks)))  
qc = QuantumCircuit(numQubits)
```

5. Appending QDCT to Quantum Circuit



```
# Add the Discrete Quantum Cosine  
self.apply1DQCT(qc, [0, 1, 2], 3)  
self.apply1DQCT(qc, [4, 5, 6], 7)
```

```
N = 2*len(dataQubitsIndex)  
  
# Symmetrize Data by entangling states with Hadamard, then  
qc.h(ancillaQubitIndex)  
for q in dataQubitsIndex:  
    qc.cx(ancillaQubitIndex, q)  
  
qftRange = list(dataQubitsIndex) + [ancillaQubitIndex]  
qc.append(QFT(len(qftRange)), qftRange)  
  
# Phase Shift so that we get the appropriately calculate  
for j, q in enumerate(qftRange):  
    qc.p((np.pi * (2**j))/(2*N), q)
```

Running the Quantum Circuit

6. Running the Quantum Circuit

```
# Compute the QDCT Result in parallel across all channels
finalState = state.evolve(qc)
```

7. Decode Qubits to Classical

```
# Decode data and reshape it to 8x8 blocks
reshaped = (np.real(finalState.data) * norm).reshape(-1, 16, 16)

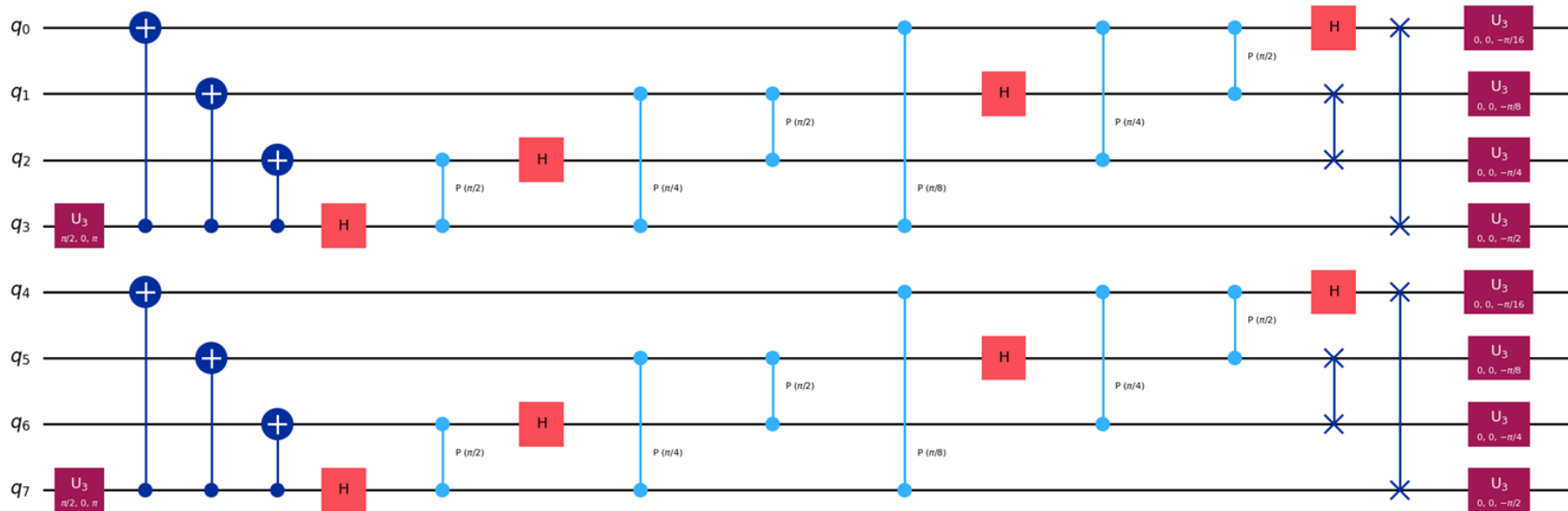
scaleMatrix = np.ones((8, 8), dtype=np.float32)

inverseSqrt2 = 1 / np.sqrt(2)

scaleMatrix[0, :] *= inverseSqrt2
scaleMatrix[:, 0] *= inverseSqrt2

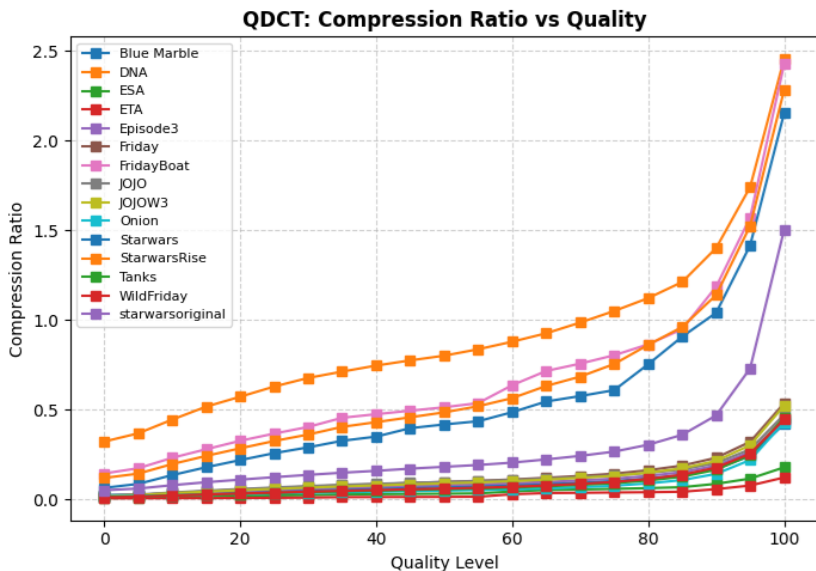
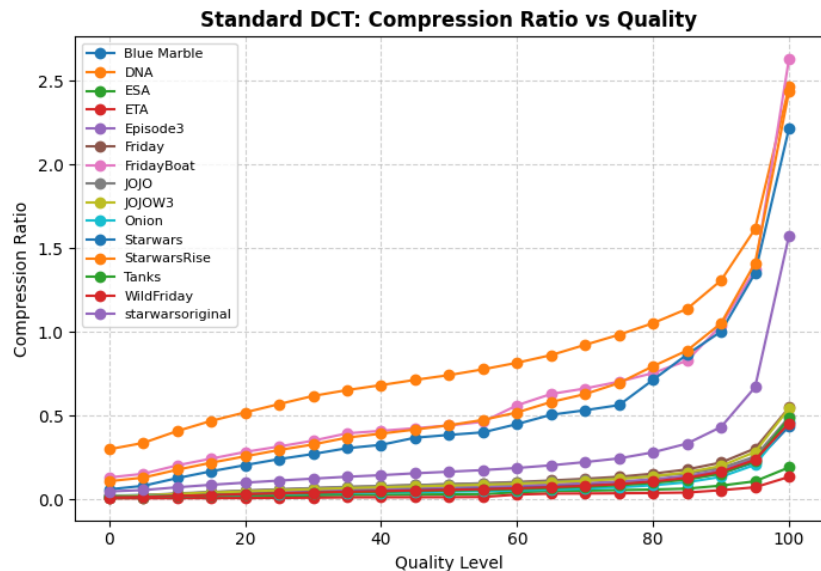
# Extract TopLeft 8x8 and return it
return reshaped[:blocks, :8, :8] * (2.0 * scaleMatrix)
```

Quantum Circuit Decomposed



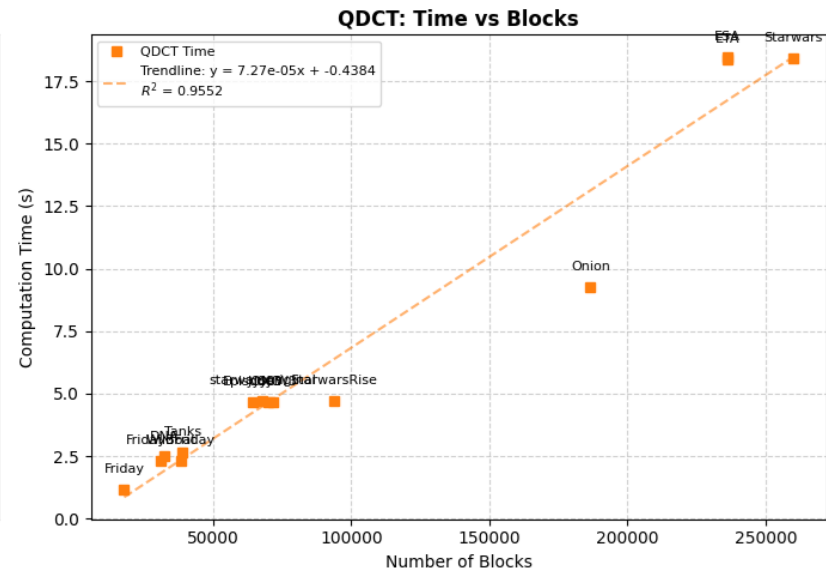
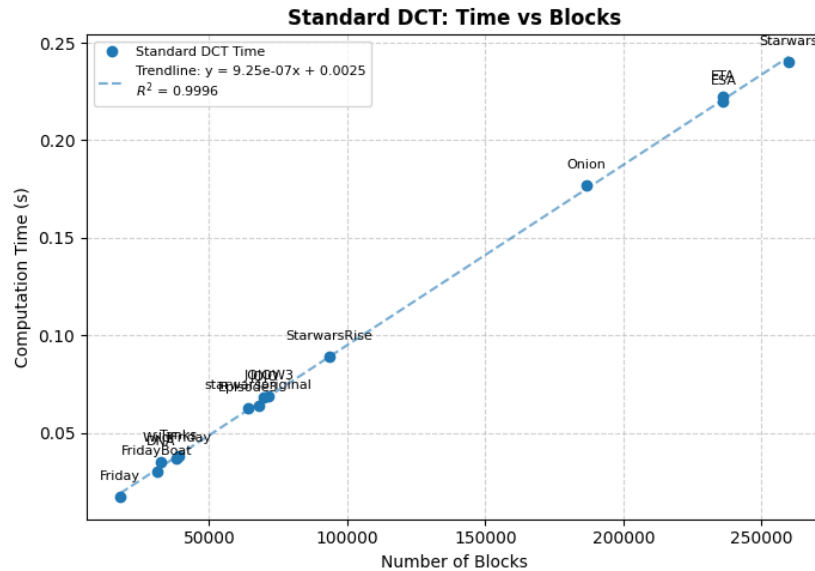
Algorithm Results

Comparison of Compression Ratio



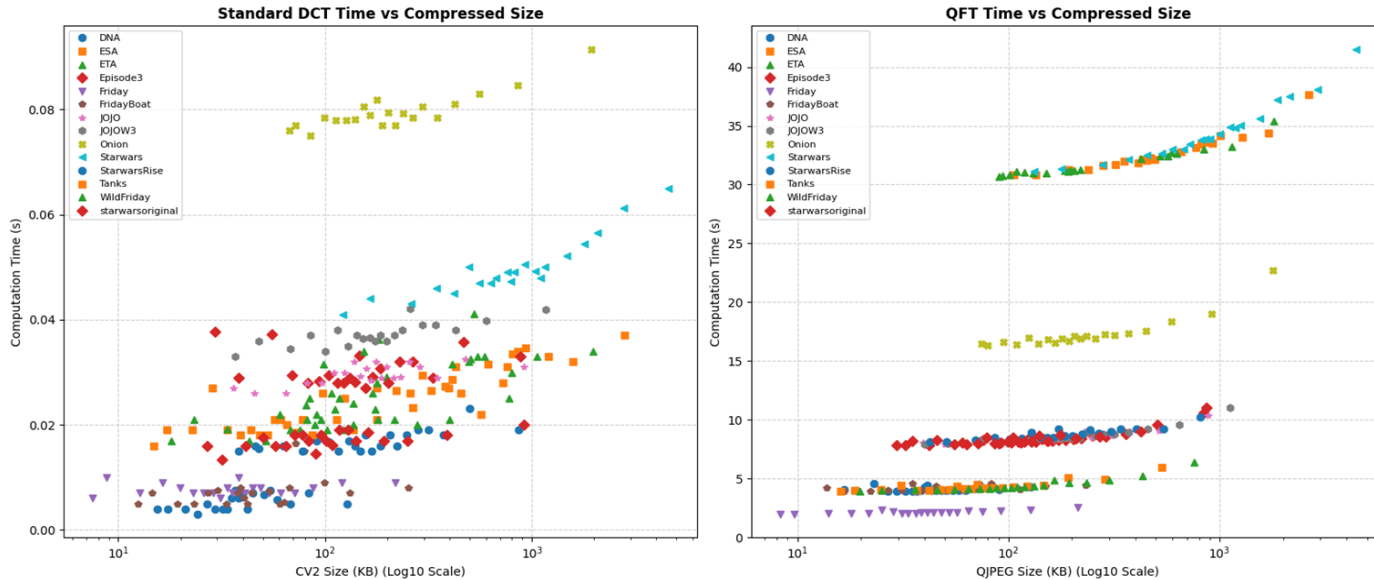
Nearly identical compression ratios
Exponential scaling near 100% quality
Ideal quality ~80%

Comparison of Compute Time



Both have linear computation relation
Comparable R-squared fit for both DCT & QDCT
Slight slope increase (x100)

Comparison of Time Vs. Comp.



Caused by byte encoding
Parabolic trends
QDCT offer consolidated trend

QUANTUM OR CLASSIC?

(Visual Confirmation)

Quantum or Classic?



Original Image



Quality = 0



Quality = 100

Method A

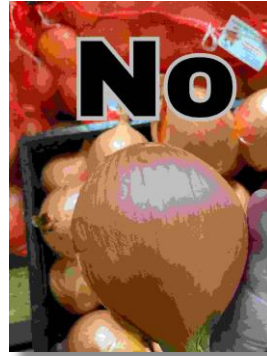


Method B

Quantum or Classic?



Original Image

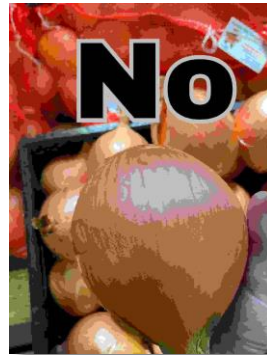


Quality = 0



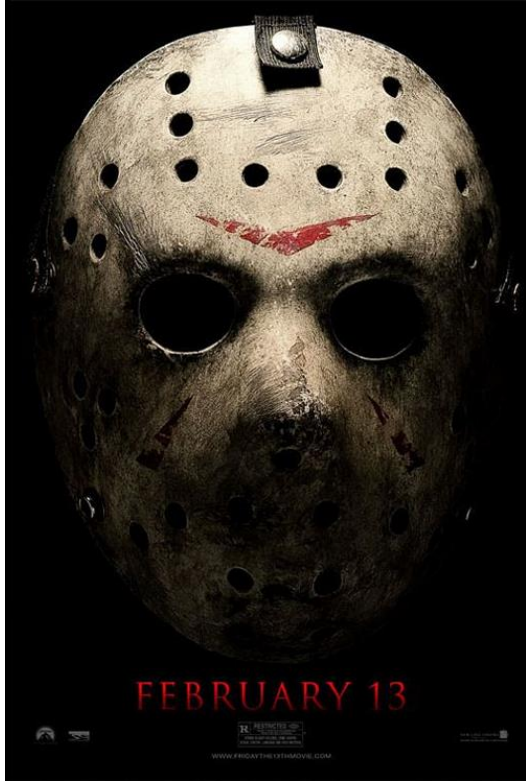
Quality = 100

Method A



Method B

Quantum or Classic?



Original Image



Quality = 0



Quality = 100

Method A



Method B

Alternative Methods & Efforts

Different QDCT Circuit [9]

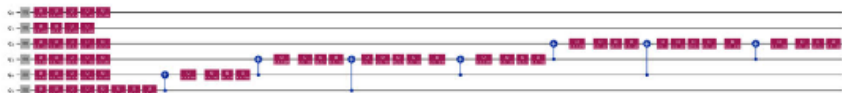


FIGURE 6. 8 x 8 QDCT decomposed circuit.

Alternative Grover Algorithm [10]

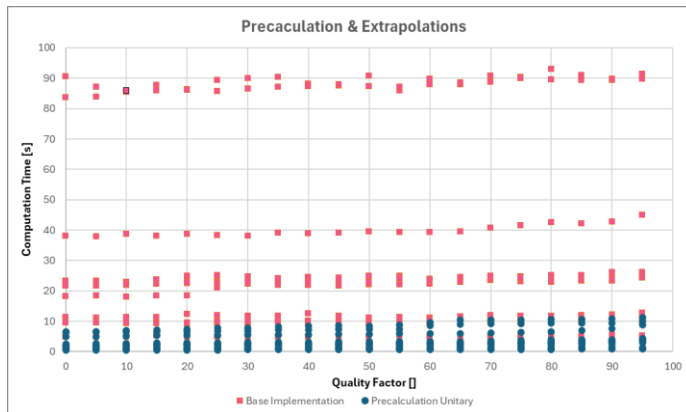
Abstract

Discrete Cosine Transform (DCT) is very important in image compression. Classical 1-D DCT and 2-D DCT has time complexity $O(N \log_2 N)$ and $O(N^2 \log_2 N)$ respectively. This paper presents a quantum DCT iteration, and constructs a quantum 1-D and 2-D DCT algorithm for image compression by using the iteration. The presented 1-D and 2-D DCT has time complexity $O(\sqrt{N})$ and $O(N)$ respectively. In addition, the method presented in this paper generalizes the famous Grover's algorithm to solve complex unstructured search problem.

Precomputation of Unitary Matrix

By doing an initial QDCT and finding the associated unitary matrix.

Extrapolate with Classical.



Conclusions

Conclusion

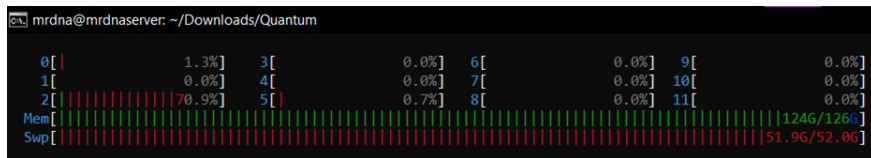
Theoretical Advantages

- QFT complexity $\rightarrow O(\log^2 N)$,
Classical FFT/DCT $\rightarrow O(N \log N)$
- Enables efficient transformation in
quantum state space

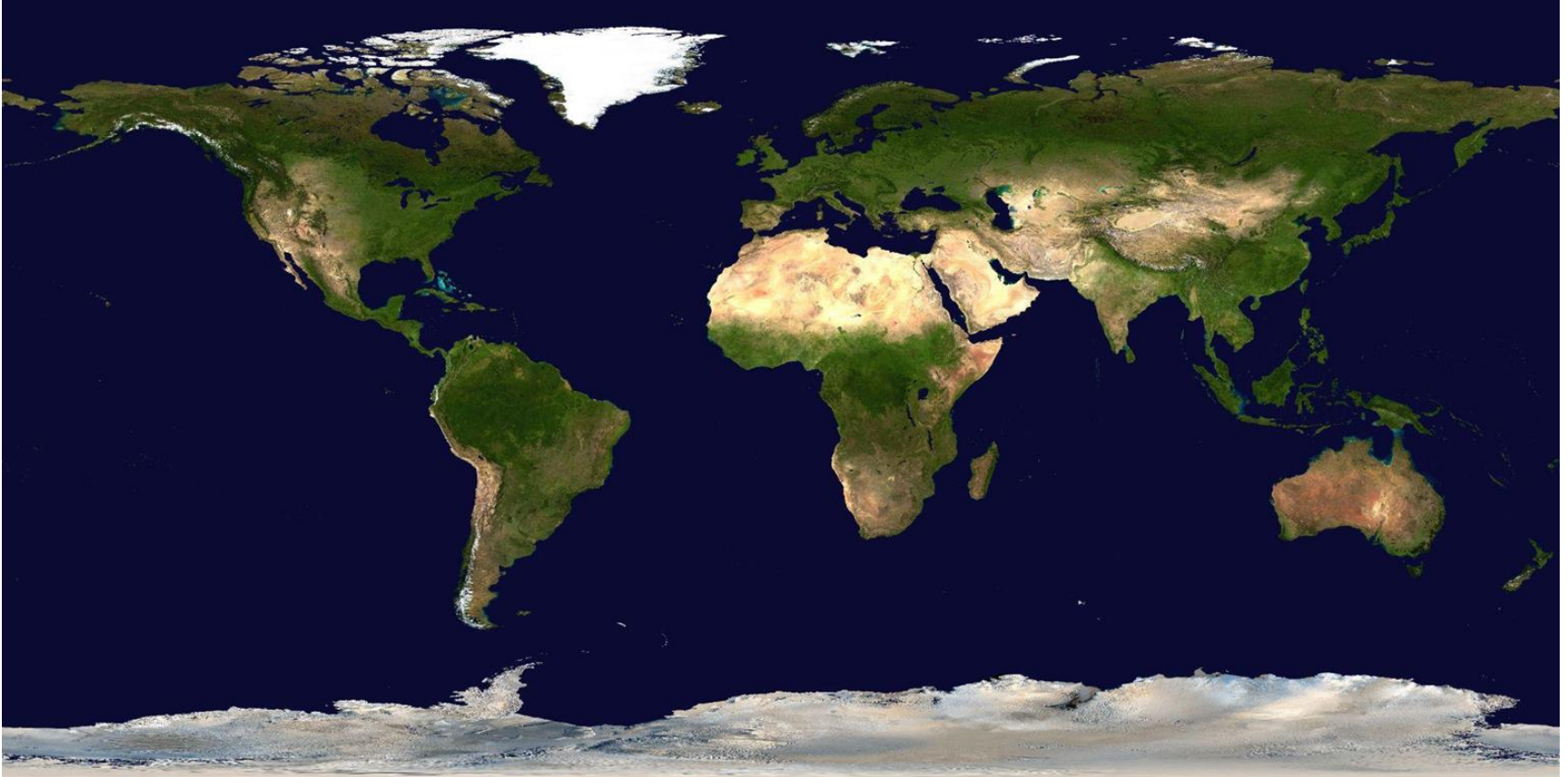
Practical Challenges

- State preparation overhead
- Measurement limitations
- Computationally expensive
- Simulation cost
- Resource constraints

★ Current implementations remain
limited by hardware and measurement
constraints



Blue Marble (NASA)



21600 x 10800 Pixels, 180 MB Initial Size, 75MB Final Size, 1100 Sec, 180GB RAM

References

- [1] https://en.wikipedia.org/wiki/Digital_photography
- [2] https://en.wikipedia.org/wiki/Image_compression
- [3] <https://en.wikipedia.org/wiki/JPEG>
- [4] https://en.wikipedia.org/wiki/Chroma_subsampling
- [5] <https://wolfcrow.com/understanding-luminance-and-chrominance/>
- [6] <https://micro.magnet.fsu.edu/primer/java/digitalimaging/processing/jpegcompression/index.html>
- [7] https://cc.ee.ntu.edu.tw/~rbwu/rapid_content/course/QC/Cleve2000.pdf
- [8] https://web.archive.org/web/20111125071212/http://dasan.sejong.ac.kr/~dihan/dip/p5_DCT.pdf
- [9] <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=11242178>
- [10] <https://arxiv.org/pdf/quant-ph/0111038>
- [11] <https://en.wikipedia.org/wiki/YCbCr>
- [12] <https://arxiv.org/pdf/quant-ph/0601043>



Thank You!

Questions?